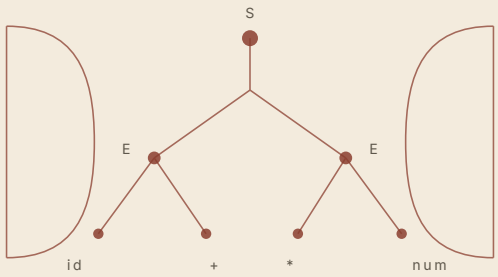
BMÜ325 · PROGRAMLAMA DİLLERİ

Sözdizimi ve Anlambilim

Gramerler, türetmeler, özellik gramerleri ve anlam tanımlama

BNF'den denotasyonel semantiğe · Biçimsel tanımlama

BÖLÜM 3 · PD3



Ders slaytlarından derlenmiştir: PD3 (136 slayt).
Metin içermeyen eğitsel slaytlar görsel olarak incelenmiştir.
Ek açıklamalar "Açıklayıcı not" etiketiyle işaretlenmiştir.

ÖN KISIM · OKUMA NOTU

Bu kitapçık hakkında

Bu kılavuz, PD3 ders destesinin sınav odaklı, kâğıt hissi veren bir çalışma özetidir. Amaç slaytları tekrar etmek değil; bir dilin sözdizimini ve anlamını biçimsel olarak nasıl tanımladığımızı, bu tanımların hangi soruları yanıtladığını ve nerede yetersiz kaldığını tek bir metinde toplamaktır.

KAYNAK	PD3 “Sentaks ve Semantiği Tanımlama” (136 slayt). Kapsam 3.1–3.5 arasındaki beş alt bölümdür.
SAYFA DÜZENİ	Kaynak sırası korunmuştur; referanslar [PD3, s. 22] biçimindedir. Slayt numarası ile PDF sayfası aynıdır.
YÖNTEM	Metin katmanı PDF’den çıkarıldı; metin içermeyen eğitsel slaytlar sayfa görüntüsü olarak okundu.
DOĞRULUK	Kaynakta bulunmayan ek bilgi yalnızca **Açıklayıcı not** etiketiyle verilir. Kaynak içi tutarsızlıklar sessizce düzeltilmez.
SÜRÜM	PD3 tek başına; aynı şablon PD4–PD8 için de kullanılacaktır.

KILAVUZUN EKSENI

Bir dil iki şeyle tanımlanır: hangi programlar geçerlidir (sözdizimi) ve geçerli program ne anlama gelir (anlambilim). PD3, birinci soruyu BNF/EBNF ve bağlam-bağımsız gramerlerle; ikinci soruyu işlemsel, aksiyomatik ve denotasyonel semantikle yanıtlar. Bu iki eksen birbirinden bağımsız değildir: gramer, anlamın taşıyıcısı olan ağacı kurar; özellik gramerleri bu ağaca tip ve tanım bilgisi ekler.

KAYNAK VE KAPSAM

PD3, Sebesta çizgisindeki sözdizimi/semantik anlatımını izler; kaynakçada Sebesta (2013), Watt (2004), Scott (2009), Evans, Nierstrasz, Bermúdez, Jha ve Türkçe ders notları sayılır [PD3, s. 136]. Kılavuz yalnızca slaytlardaki bilgiyi düzenler; slaytta bulunmayan yeni bir formalizm veya tarih eklemez. Ödev slaytı (s. 135) kapsam dışıdır; yalnızca G bölümünde not edilir.

KISALTMALAR

PD programlama dili ders destesi · **BNF** Backus–Naur formu · **EBNF** genişletilmiş BNF · **CFG** bağlam-bağımsız gramer (context-free grammar) · **AG** özellik grameri (attribute grammar) · **SOS** yapısal işlemsel semantik (structured operational semantics) · **PDA** aşağı sürüklemeli otomat (pushdown automaton) · **RVM** Register Virtual Machine · **PC** program sayacı (program counter) · **AST** soyut sözdizimi ağacı (abstract syntax tree).

İÇİNDEKİLER

Kılavuzun haritası

Sayfa numaraları bu PDF'e aittir; kaynak slayt referansları metin içinde verilmiştir.

A · ÇERÇEVE

Kılavuzun ekseni 2

B · AYRINTILI İNCELEME

Bölüm 3 — Sözdizimi ve anlambilim 4

3.1 Giriş 4

3.2 Genel sözdizimi tanımlama problemi 5

3.3 Sözdizimi tanımlamanın biçimsel yöntemleri 6

3.4 Özellik (Attribute) Gramerleri 8

3.5 Semantik (Anlambilim) 9

C–G · ORTAK BÖLÜMLER

C. Kavramlar ve tanımlar 14

D. Yöntemler ve süreçler 15

E. Formüller, kurallar ve olgular 16

F. Bağlantılar ve karşılaştırmalar 18

G. Kapsam denetimi 19

B · AYRINTILI İNCELEME
KAYNAK: PD3 · 136 SLAYT

BÖLÜM 3

Sözdizimi ve Anlambilim

Bir programın sözdizimsel olarak doğru olması, anlamlı olduğu anlamına gelir mi? BNF ile tanımlanan bir gramer neden tek başına yetmez? Özellik gramerleri tipe ve tanıma nasıl bağlanır? Bir programın anlamı makine durumuyla, mantıksal ön-son koşullarla veya matematiksel fonksiyonlarla nasıl tanımlanır?

Kaynak aralığı: PD3, s. 1–136 · Alt bölümler: 3.1–3.5

3.1 Giriş

Her programlama dilinde geçerli programları belirleyen bir dizi kural vardır. Bu kurallar ikiye ayrılır: **sentaks (sözdizimi, syntax)** ve **semantik (anlambilim, semantics)**. Her deyimin sonunda noktalı virgül bulunması bir sözdizimi kuralı; bir değişkenin kullanılmadan önce tanımlanması bir anlambilim kuralıdır [PD3, s. 3].
Sözdizimi, ifadelerin (statements), deyimlerin (expressions) ve program birimlerinin **biçimini veya yapısını** tanımlar. **Anlambilim**, aynı birimlerin **anlamını** tanımlar. Sözdizimi ve anlambilim birlikte bir dilin tanımını sağlar; bu tanımın kullanıcıları diğer dil tasarımcıları, uygulamacılar (implementers) ve programcılardır [PD3, s. 4–5].

TANIM

Sözdizimi (syntax): Dilin geçerli deyim ve program birimlerinin biçimsel yapısını belirleyen kurallar.
Anlambilim (semantics): Sözdizimsel olarak geçerli birimlerin ne anlama geldiğini belirleyen kurallar. [PD3, s. 4]

KÜÇÜK FARK, BÜYÜK ANLAM

Sözdizimi ile anlam arasındaki farkı programlama dillerinden bağımsız olarak görmek için tarih gösterimi kullanılabilir: `gg.aa.yyyy` biçimi bir sözdizimidir; bu biçimin hangi günü gösterdiği ise anlamdır. Programlamada küçük sözdizimi farkları büyük anlam farkları doğurabilir [PD3, s. 6]:

```
while (i < 10) { a[i] = ++i; }   // önce i artır, sonra a[i]'ye yaz
while (i < 10) { a[i] = i++; }   // önce a[i]'ye yaz, sonra i artır
```

İki deyim de sözdizimsel olarak geçerlidir; fark, **değerlendirme sırası** ve dolayısıyla anlamdır. Bu örnek, “gramer doğru” ile “program doğru” arasındaki boşluğu gösterir.

SOYUT SÖZDİZİMİ

Bir dilin **soyut sözdizimi**, o dildeki her yapının **anlamlı bileşenlerini** tanımlar. `+ab` (prefix), `a+b` (infix) ve `ab+` (postfix) gösterimlerinde `+` işlemcisi ile `a` ve `b` alt ifadeleri aynı anlamlı bileşenlerdir; bu nedenle üçünün de ağaç gösterimi aynıdır [PD3, s. 7].
Bir ifadedeki işlemci/işlenen yapısını gösteren ağaçlara **soyut sözdizimi ağaçları** denir. Bu ağaçlar, ifadenin yazıldığı gösterimden bağımsız olarak sözdizimsel yapıyı gösterebildikleri için bu adı alır. Uygun işlemciler geliştirilerek diğer yapılar için de genişletilebilirler; örneğin `if a <= b then a else b` ifadesi de ağaçla gösterilebilir [PD3, s. 8].

METİNSEL SÖZDİZİMİ

Hem doğal diller hem programlama dilleri bir alfabedeki karakter dizilerinden oluşur. Bir dilin karakter dizilerine **cümle** veya **deyim** denir; sözdizim kuralları, alfabeden hangi karakter dizilerinin o dile ait olduğunu belirler. En büyük ve karmaşık programlama dili bile sözdizimsel olarak oldukça basittir [PD3, s. 9].
Bir programlama dilindeki en düşük düzeyli sözdizimsel birimlere **lexeme** denir; programlar karakter yerine lexeme’ler dizisi olarak düşünülebilir. Lexeme’lerin gruplanmasıyla dile ilişkin **token**’lar oluşur. Bir tanımlayıcı token’ının `toplam` veya `sonuc` gibi lexeme’leri olabilir; `+` işlemcisinin ise tek olası lexeme’i vardır. Boşluk, sekme veya yeni satır karakterleri token’lar arasına yerleştirildiğinde programın anlamı değişmez [PD3, s. 9–10].

C DEYIMINDE LEXEME VE TOKEN

`int x;` deyiminde `int` bir **keyword**, `x` bir **identifier (ID)**, `;` bir **symbol** token’ıdır. `cin >> x;` satırında `cin` ID, `>>` sembol, `x` ID, `;` semboldür. Token’lar bir **değer/tip ikilisi** olarak düşünülebilir [PD3, s. 17–18].

KRITIK AYRIM

Soyut sözdizimi / metinsel sözdizimi: Soyut sözdizimi anlamlı bileşenlerin ağacını; metinsel sözdizimi bu ağacın hangi karakter dizileriyle yazılacağını tanımlar. `a+b`, `+ab` ve `ab+` aynı soyut ağaca sahiptir; metinsel gösterimleri farklıdır. Gramerler metinsel sözdizimini tanımlar, ayrıştırıcı bu metinden soyut yapıyı kurar.

3.2 Genel sözdizimi tanımlama problemi

Sözdizimi tanımlamanın temel terimleri şunlardır [PD3, s. 11]:

- **Cümle (sentence):** Herhangi bir alfabedeki karakterlerden oluşan string.
- **Dil (language):** Cümlelerden oluşan küme.
- **Lexeme:** Bir dilin en alt seviyedeki sözdizimsel birimi (`*`, `sum`, `begin`).
- **Token (simge):** Lexeme'lerin kategorisi (örneğin **identifier**).

SÖZCÜKSEL ANALİZ

Sözcüksel analiz (lexical analysis, tarama), karakter akışını token akışına çevirir. `val = 10 * val + i` dizisi şu token'lara ayrılır [PD3, s. 12]:

```
Karakter akışı:  v a l   =   1 0   *   v a l   +   i
Token numarası:    1     3    2   4     1     5   1
Token akışı:    (ident) (assign) (number) (times) (ident) (plus) (ident)
Token değeri:    "val"     -       10       -     "val"    -    "i"
```

ÖNİŞLEMCİ VE SÖZCÜKSEL ANALİZ

Önişlemci, kaynak dosya üzerinde makro genişletme ve yorum silme gibi dönüşümler yapar; değiştirilmiş bir kaynak dosya üretir. Örneğin `#define MAX 50` ve `#define MIN 10` makroları, `x = MAX;` ve `x = y - MIN;` satırlarını `x = 50;` ve `x = y - 10;` hâline getirir [PD3, s. 13]. Sözcüksel analiz ise değiştirilmiş kaynak dosyayı okuyup token listesi üretir; iki aşama birlikte de çalışabilir [PD3, s. 14–15].

```
// Girdi
void main() { int x; x = 50; int y; x = y - 10; }

// Sözcüksel analiz çıktısı (token listesi)
void  main  (  )  {  int  x  ;  x  =  50  ;  int  y  ;  x  =  y  -  10  ;  }
keyword ID  symbol symbol symbol keyword ID symbol ID assign number symbol keyword ID symbol ID assign ID
symbol number symbol
```

`>>` BELİRSİZLİĞİ

`Array<Array<int>> someArray;` yazımında `>>` iki kapanış açısı olarak mı yoksa sağa kaydırma işlemcisi olarak mı okunmalıdır? Sözcüksel analiz en uzun eşleşmeyi seçtiğinde `>>` tek bir token olur ve iç içe şablon kapanışı bozulur. Çözüm, araya boşluk koymaktır: `Array<Array<int> > someArray;` [PD3, s. 19]. Bu örnek, sözcüksel düzeydeki bir kararın sözdizimini nasıl etkilediğini gösterir.

TANIYICILAR VE ÜRETEÇLER

Dillerin biçimsel tanımı iki araçla yapılır [PD3, s. 20–21]:

- **Tanıyıcılar (recognizers):** Bir dilin girdi stringlerini okur ve stringin dile ait olup olmadığına karar verir. Derleyicinin sözdizim analizi bir tanıyıcıdır; sonlu otomatlar örnektir. `F = (Q, Σ, δ, q0, F)` geçiş diyagramında `((00)*1(11)*)+0` deseni `001110` ve `111110` dizilerini kabul eder, `000110` dizisini reddeder.
- **Üreteçler (generators):** Bir dilin cümlelerini üretir. Düzenli ifadeler ve bağlam-bağımsız gramerler üreteçtir; bir cümlenin sözdizimsel doğruluğu, üretecin yapısıyla karşılaştırılarak anlaşılır.

AÇIKLAYICI NOT

Tanıyıcı "bu string dilde mi?" sorusunu **karar** eder; üreteç "dilin cümlelerini" **üretir**. Aynı dil için ikisi de kullanılabilir; derleyici pratiğinde tanıyıcı (parser), dil tanımında ise üreteç (gramer) öne çıkar.

HATA TÜRLERİ

Sözcüksel ve sözdizimsel hatalar farklı aşamalarda yakalanır [PD3, s. 26]:

HATA TÜRÜ	ÖRNEK	AŞAMA
Sözcüksel / token hatası	`int x$y;`, `int 32xy;`, `45b`, `45ab`, `x = x @ y;`	Sözcüksel analiz
Sözdizim hatası	`X = ;`, `Y = x +;`, `Z = [;`	Sözdizim analizi
Anlambilim hatası	`x = "SHERRY";` (x tamsayıysa)	Anlam analizi

Derleme hattı bu ayrımı görünür kılar: kaynak program → sözcüksel analiz → token'lar → sözdizim analizi → ayrıştırma ağacı → anlam analizi → ara temsil → eniyileme → kod üreteci → hedef makine kodu. **Sembol tablosu** bu aşamalara paralel çalışır [PD3, s. 27–29].

3.3 Sözdizimi tanımlamanın biçimsel yöntemleri

Bir veya daha çok dilin sözdizimini anlatmak için kullanılan dile **metadil (metalanguage)** denir. Programlama dillerinin sözdizimini anlatmak için **BNF (Backus–Naur Form)** kullanılır; buna karşılık anlam tanımlamak için BNF kadar yaygın tek bir metadil yoktur [PD3, s. 22].

BAĞLAM-BAĞIMSIZ GRAMER

Gramer, bir programlama dilinin metinsel (somut) sözdizimini açıklamak için kullanılan gösterimdir; anahtar kelimelerin ve noktalama işaretlerinin yerleri de dahil olmak üzere bir dizi kuraldan oluşur [PD3, s. 23]. **Bağlam-bağımsız gramer (CFG)**, Noam Chomsky tarafından 1950'lerin ortasında geliştirildi; dil üreteçleri doğal dillerin sözdizimini tanımlamak için kullanıldı. CFG'ler `A → γ` biçiminde gösterilir; γ terminal ve terminal olmayan sembollerden oluşabilir. Bu diller **aşağı sürüklemeli otomatlar (PDA)** tarafından kabul edilir ve hemen hemen bütün programlama dillerinin temelini oluşturur [PD3, s. 30].

BNF'NİN DÖRT BİLEŞENİ

BNF, John Backus tarafından ALGOL 58'i belirlemek için icat edildi; Peter Naur ALGOL 60 için biraz değiştirdi ve adı **Backus–Naur Form** oldu. BNF, CFG'ye eşdeğer bir metadildir; soyutlamalar sözdizimsel yapı sınıflarını temsil eder ve **terminal olmayan semboller** olarak davranır [PD3, s. 31].

#	BİLEŞEN	TANIM / ÖRNEK
1	**Terminal semboller**	Bir dilde geçerli yapıları oluşturmak için birleştirilen, daha alt parçalara ayrılamayan atomik semboller: `+`, `*`, `-`, `%`, `if`, `>=`.
2	**Terminal olmayan semboller**	Dilin kendisinde bulunmayan, kurallarla tanımlanan ara tanımlar; `< >` arasında yazılır: `<Statement>`, `<Expr>`, `<Type>`.
3	**Kurallar**	Bir terminal olmayanın bileşenlerini tanımlar: sol taraf terminal olmayan, `:=` veya `→`, sağ tarafta terminal/terminal olmayan dizisi.
4	**Başlangıç sembolü**	Dilin ana elemanını gösteren, terminal olmayanlardan seçilen amaç sembol.

Kural örnekleri [PD3, s. 34–37]:

```
<ident list> → identifier | identifier, <ident list>
<if_stmt>    → if <logic_expr> then <stmt>
<stmt>       → <single_stmt>
             | begin <stmt_list> end
<tanımlayıcı_listesi> := tanımlayıcı | tanımlayıcı, <tanımlayıcı_listesi>
```

`|` sembolü "veya" anlamındadır; bir soyutlama birden fazla sağ tarafa (RHS) sahip olabilir. Bir kuralın sol tarafı sağ tarafta yer alıyorsa kural **özyinelemelidir**. Bir gramer, kuralların boş olmayan sonlu bir kümesidir [PD3, s. 35–37].

TÜRETME

BNF ile bir dildeki cümleler, başlangıç sembolünden başlayarak kuralların sırayla uygulanmasıyla oluşturulur; buna **türetme (derivation)** denir. Türetme, başlangıç sembolüyle başlayan ve yalnızca terminal sembollerden oluşan bir cümleyle biten kuralların tekrarlamalı uygulamasıdır [PD3, s. 38].
Örnek gramer [PD3, s. 43]:

```
<program> → <stmts>
<stmts>   → <stmt> | <stmt> ; <stmts>
<stmt>    → <var> = <expr>
<var>     → a | b | c | d
<expr>    → <term> + <term> | <term> - <term>
<term>    → <var> | const
```

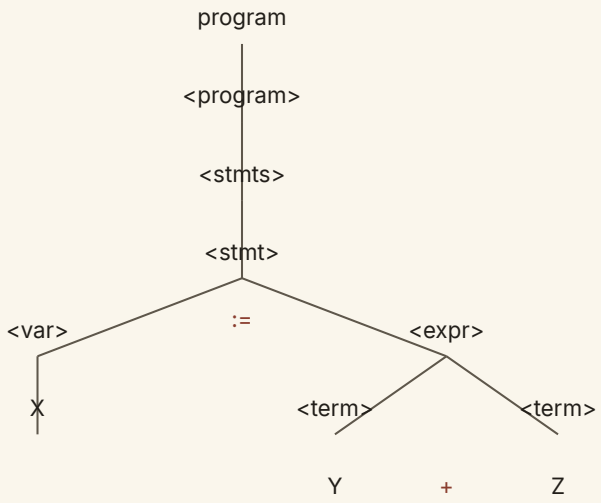
Bu gramerle `a = b + const` türetmesi [PD3, s. 44]:

```
<program> => <stmts> => <stmt> => <var> = <expr>
          => a = <expr> => a = <term> + <term>
          => a = <var> + <term> => a = b + <term>
          => a = b + const
```

Türetmede yer alan bütün sembol stringleri **cümlesel biçim (sentential form)** dir. Yalnızca terminal semboller içeren cümlesel biçim bir **cümle (sentence)** dir. Her adımda **en soldaki** terminal olmayan genişletiliyorsa **ensol türetme (leftmost derivation)**; en sağdaki genişletiliyorsa **ensağ türetme** denir; bazı türetmeler ikisi de değildir [PD3, s. 45].

AYRIŞTIRMA AĞACI

Gramerler, tanımladıkları dilin cümlelerinin **hiyerarşik sözdizimsel yapısını** tarif eder; bu yapılara **ayrıştırma ağacı (parse tree)** denir. Ağacın en alt düğümlerinde terminal semboller, diğer düğümlerde terminal olmayanlar bulunur. Ayrıştırma ağaçları ile türetmeler birbirinden türetilebilir [PD3, s. 48–49].



PD3, s. 48–49 · X := Y + Z

ŞEKİL 3.1 X := Y + Z deyiminin ayrıştırma ağacı. Kök <program>, dallar <stmts>, <stmt>, <var> ve <expr>; yapraklar terminal sembollerdir. [PD3, s. 48–49]

BELİRSİZLİK

Bir gramer, ancak ve ancak **iki veya daha fazla farklı ayrıştırma ağacı** olan bir cümlesel biçim üretiyorsa **belirsizdir (ambiguous)** [PD3, s. 51]. Örnek [PD3, s. 52]:

```
<expr> → <expr> <op> <expr> | const
<op>   → / | -
```

`const - const / const` dizisi iki farklı ağaçla türetilebilir: biri çıkarmayı, diğeri bölmeyi önce yapar. Operatör önceliğini gösteren kurallarla belirsizlik giderilir [PD3, s. 53]:

```
<expr> → <expr> - <term> | <term>
<term> → <term> / const | const
```

Bu gramerde bölme, çıkarmadan daha derinde olduğu için önce değerlendirilir. **Birleşirlik (associativity)** de gramerle gösterilir: `<expr> → <expr> + <expr> | const` belirsizdir; `<expr> → <expr> + const | const` belirli ve sol birleşirlidir [PD3, s. 54].

> KRITIK AYRIM
> **Belirsiz dil / belirsiz gramer:** Belirsizlik dilin değil **gramerin** özelliğidir. Aynı dil, belirsiz bir gramerle de belirsiz olmayan bir gramerle de tanımlanabilir. Sınavda "bu dil belirsiz midir?" yerine "bu gramer belirsiz midir?" sorusu soruluyorsa, verilen kurallar kümesine bakılır.

GENİŞLETİLMİŞ BNF (EBNF)

EBNF, BNF'in okunabilirliğini ve yazılabilirliğini artırmak için eklenmiş üç özellik sunar [PD3, s. 55–59]:

ÖZELLİK	GÖSTERİM	ANLAM
Seçimlik (optionality)	`[ ]`	İçindeki bölüm hiç yer almayabilir veya bir kez bulunabilir.
Yineleme (repetition)	`{ }`	İçindeki bölüm sıfır veya daha fazla kez yinelenebilir.
Değiştirme (alternation)	`( ... | ... )`	

Örnekler [PD3, s. 56–58]:

```
<proc_call> → ident [ "(" <expr_list> ")" ]
<term>      → <term> ( "+" | "-" ) const
<ident>     → letter { letter | digit }
dogal sayi  ::= sifir haric sayi , { sayi }
```

`[ ]`, `{ }` ve `|` EBNF'de gösterimi kısaltan **metasembollerdir** [PD3, s. 60]. BNF ile EBNF arasındaki fark, aynı dilin daha az kuralla yazılmasıdır [PD3, s. 61]:

```
# BNF
<expr> → <expr> + <term> | <expr> - <term> | <term>
<term> → <term> * <factor> | <term> / <factor> | <factor>

# EBNF
<expr> → <term> { ("+" | "-") <term> }
<term> → <factor> { ("*" | "/") <factor> }
```

ARİTMETİK İFADELERİN GRAMERİ

Kaynak, bir gramerin bileşenlerini aritmetik ifadeler üzerinden özetler [PD3, s. 62]:

```
Expr   = [ "+" | "-" ] Term { ( "+" | "-" ) Term }.
Term   = Factor { ( "*" | "/" ) Factor }.
Factor = ident | number | "(" Expr ")".
```

- **Terminal semboller:** `+`, `-`, `*`, `/`, `(`, `)` (tek örnekli).
- **Terminal sınıfları:** `ident`, `number` (çoklu örnekler).
- **Terminal olmayan semboller:** `Expr`, `Term`, `Factor`.
- **Başlangıç sembolü:** `Expr`.

SÖZDİZİM GRAFİKLERİ

BNF ve EBNF kuralları, ayrıştırma ağacı dışında **sözdizim grafikleri (syntax graphs)** ile de gösterilebilir. İlk olarak Pascal'ın gramerini açıklamakta kullanılmıştır ve BNF'ye eşdeğerdir. Kurallar, düğümleri semboller olan iki yönlü yönlendirilmiş grafiklerle gösterilir; grafikteki olası yollar, kuraldaki terminal olmayanı tanımlayan olası sembol sıralarını verir. Terminal semboller **oval**, terminal olmayanlar **dikdörtgen** düğümlerle gösterilir [PD3, s. 75–77].

3.4 Özellik (Attribute) Gramerleri

Bağlam-bağımsız gramerler bir programlama dilinin **bütün sözdizimini** tanımlayamaz. Ayrıştırma ağaçlarına bazı anlambilim bilgileri eklenmelidir; ancak her şeyi gramere koymak ağacı aşırı büyütür. Tipik örnekler **tip uyumluluğu** ve bazı dillerde **değişkenlerin kullanılmadan önce tanımlanması** zorunluluğudur [PD3, s. 63]. **Özellik gramerlerinin (attribute grammars, AG)** birincil değerleri **statik semantik belirtimi** ve **derleyici tasarımı** (statik semantik denetimi)dir [PD3, s. 63].

TANIM

Bir özellik grameri `G = (S, N, T, P)`, aşağıdaki eklemelerle birlikte bir CFG'dir [PD3, s. 64]:

- Her gramer sembolü `x` için özellik değerlerinden oluşan bir `A(x)` kümesi vardır.

— Her kural, içindeki terminal olmayanların belirli özelliklerini tanımlayan bir **fonksiyonlar kümesine** sahiptir.

— Her kural, özellik tutarlılığını denetlemek için **karşılaştırma belirtimlerinden (predicate)** oluşan (boş olabilen) bir kümeye sahiptir.

`X0 → X1 ... Xn` kuralında [PD3, s. 65]:

— `S(X0) = f(A(X1), ..., A(Xn))` biçimindeki fonksiyonlar **sentezlenmiş özellikleri (synthesized)** tanımlar.

— `I(Xj) = f(A(X0), ..., A(Xn))` (1 ≤ j ≤ n) biçimindeki fonksiyonlar **miras alınmış özellikleri (inherited)** tanımlar.

— Başlangıçta yapraklarda **yerleşik özellikler** vardır.

TİP DENETİMİ ÖRNEĞİ

Kaynak, tip denetimini özellik grameriyle şöyle modeller [PD3, s. 66–71]:

```
<assign> → <var> = <expr>
<expr>   → <var> + <var> | <var>
<var>    → A | B | C
```

— `actual_type`, `<var>` ve `<expr>` üzerinde **sentezlenir**; değeri sembol tablosundan `look-up` ile bulunur.

— `expected_type`, `<expr>` üzerinde **miras alınır**; `<assign>` kuralında `<var>.actual_type` değerinden gelir.

— Kural: `<expr>.actual_type ← <var>[1].actual_type`; predicate: `<var>[1].actual_type == <var>[2].actual_type`.

— Aritmetik toplama kuralında iki operand da `int` ise sonuç `int`, değilse `real` olur; predicate `<expr>.actual_type == <expr>.expected_type` ile uyum denetlenir.

`A = A + B` cümlesinin değerlendirilmesi [PD3, s. 71–72]:

1. `A` ve `B` için `actual_type` sembol tablosundan bulunur.
2. `A`'nın tipi `expected_type` olarak `<expr>`'ye miras verilir.
3. Toplama kuralı `actual_type`'ı hesaplar (`int` veya `real`).
4. Predicate, beklenen ve gerçek tipi karşılaştırır; sonuç `TRUE` veya `FALSE` olur.

ADA PROSEDÜR ADI ÖRNEĞİ

Ada'da bir prosedürün `end` üzerindeki adının prosedür adıyla aynı olması kuralı, özellik grameriyle şöyle modellenir [PD3, s. 67]:

```
<proc_def> → procedure <proc_name>[1] <proc_body> end <proc_name>[2]
```

Semantik kural: `<proc_name>[1].string = <proc_name>[2].string`. Burada `string` bir özelliktir; kural, iki oluşumun aynı metni taşımasını zorunlu kılar.

DEĞERLENDİRME SIRASI

Özellik değerlerinin hesaplanma sırası, özellik türlerine bağlıdır [PD3, s. 69–70]:

— Bütün özellikler **miras alınmışsa** ağaç **yukarıdan aşağıya (top-down)** düzenlenir.

— Bütün özellikler **sentezlenmişse** ağaç **aşağıdan yukarıya (bottom-up)** düzenlenir.

— Çoğu zaman iki tür birlikte kullanılır; aşağıdan yukarıya ve yukarıdan aşağıya düzenlerin **kombinasyonu** gerekir.

> **KRITIK AYRIM**
>
> **Sentezlenmiş / miras alınmış:** Sentezlenmiş özellik çocuklardan ebeveyne **yukarı** akar; miras alınmış özellik ebeveynden çocuklara **aşağı** akar. `actual_type` sentezlenmiş, `expected_type` miras alınmıştır; bu yüzden tip denetimi iki yönlü bir hesaplama gerektirir.

3.5 Semantik (Anlambilim)

Sözdizimsel olarak doğru olan bütün programların bir anlamı olmayabilir. Bir programın çalıştırılabilmesi için hem sözdizimi hem anlam açısından doğru olması gerekir. Bir dilin **anlam (semantik) kuralları**, sözdizimsel olarak geçerli herhangi bir programın anlamını belirler. Sözdizimi BNF gibi ağaçlarla kolayca tanımlanabilirken anlam tanımlaması farklıdır; **semantiği tanımlamak için yaygın kabul edilmiş tek bir gösterim veya formalizm yoktur** [PD3, s. 78].

ANLAM TANIMLAMA YÖNTEMLERİ

Anlamsal tanımlama için var olan yöntemler oldukça karmaşıktır ve hiçbiri BNF kadar yaygın kullanıma ulaşmamıştır. Dil yapılarının Türkçe gibi bir doğal dilde açıklanması mümkündür; ancak doğal dil açıklamaları **açık ve kesin olmaz**. Resmî anlam tanımlama için kullanılacak metadil, iyi anlaşılmış **sayısal kavramlara** dayanmalıdır [PD3, s. 79–81]. Üç yaklaşım vardır: **işlemsel (operational)**, **kurala dayalı (axiomatic)** ve **fonksiyonel (denotational)** semantik.

İŞLEMSEL (OPERATİONAL) SEMANTİK

İşlemsel semantik, bir programı benzetim veya gerçek olarak makine üzerinde çalıştırarak anlamını açıklar. Makinenin durumundaki değişme (bellek, saklayıcılar/registers) ifadenin anlamını tanımlar. Yüksek düzeyli bir dil için **sanal makine** gerekir; donanımda saf yorumlayıcı çok pahalı olur. Yazılım saf yorumlayıcının sorunları, bilgisayara özgü ayrıntıların anlaşılmayı zorlaştırması ve tanımın **makine bağımlı** olmasıdır [PD3, s. 82].
Daha iyi alternatif, tam bir bilgisayar benzetimidir: kaynak kodunu idealleştirilmiş bir bilgisayarın makine koduna çeviren bir **çevirmen** ve o bilgisayar için bir **simülatör** oluşturulur. İşlemsel semantik, dil el kitaplarında **informal** olarak iyi işler; **formal** kullanımda aşırı karmaşıktır (örneğin VDL, PL/I'ın semantiği için kullanılmıştır) [PD3, s. 83].



ŞEKİL 3.2 İşlemsel semantiğin gerçek dünya–soyut makine eşlemesi. Program giriş fonksiyonuyla başlangıç konfigürasyonuna, geçiş kurallarıyla ara konfigürasyonlara, çıkış fonksiyonuyla cevaba dönüşür. [PD3, s. 84]

Yapısal işlemsel semantik (SOS) bir dil için beş-tuple'dır [PD3, s. 85]:

```
C : Soyut makine için konfigürasyon kümesi
⇒ : Geçiş bağıntısı (C × C'nin alt kümesi)
I : Program → C  (giriş fonksiyonu)
F : Final konfigürasyonlar kümesi
O : F → Cevap  (çıkış fonksiyonu)
```

Örnek soyut makine **RVM (Register Virtual Machine)**, konfigürasyonu komutlar dizisi, program sayacı (PC) ve kaydedici değerleriyle tanımlar: `C = Komutlar × PC × RegisterFile` [PD3, s. 86]. Giriş fonksiyonu, `0`'dan `n-1`'e kadar numaralı `n` komutlu program için `Komutlar[m] = Program[m]`, `PC = 0`, `RegisterFile[n] = 0` (tüm n tamsayılar için) biçimindedir; aksi hâlde `Komutlar[m] = ERROR` olur. Final konfigürasyonlarda `Komutlar[PC] = HALT`; çıkış, `RegisterFile[0]`'daki değerdir [PD3, s. 87–88].
İşlemsel anlam, bir ifadeyi daha kolay anlaşılır bir dile çevirerek de açıklanabilir [PD3, s. 90–91]:

```
// C/Java for döngüsü
for (expr1; expr2; expr3) { ... }

// İşlemsel anlam (ara dil)
expr1;
loop: if expr2 == 0 goto out;
      ...
      expr3;
      goto loop;
out:  ...
```

KURALA DAYALI (AXİOMATİC) SEMANTİK

Aksiyomatik semantik **biçimsel mantığa (predicate calculus)** dayanır. Orijinal amaç **biçimsel program doğrulaması**dır: dildeki her ifade tipi için **aksiyomlar** veya **çıkarsama kuralları** tanımlanır; deyimlere **iddia (assertion)** denir [PD3, s. 92].

Bir ifadenin önündeki iddia **önşart (precondition)**, arkasındaki iddia **sonşart (postcondition)** olur. **En zayıf önşart (weakest precondition)**, sonşartı garanti eden asgari kısıtlayıcı önşarttır [PD3, s. 93].
Ön-son biçimi: `{P} ifade {Q}`. Örnek: `a = b + 1 {a > 1}` için mümkün bir önşart `{b > 10}`, en zayıf önşart `{b > 0}`'dır [PD3, s. 94].
Atama kuralı: `X = E` ve sonşart `Q` ise önşart `P = {Q_{x→E}}` olur; yani `Q` içindeki bütün `X`'ler `E` ile değiştirilir [PD3, s. 95]:

```
{Q_{x→E}}  x = E  {Q}

x = 2*y - 3  {x > 25}
2*y - 3 > 25  →  y > 14
En zayıf önşart: {y > 14}
```

`{x > 5} x = x - 3 {x > 0}` örneğinde `x - 3 > 0` → `x > 3`; `{x > 5}` önşartı `{x > 3}`'ü ima ettiği için doğruluk kanıtlanır [PD3, s. 96]. **Sonuç (consequence) kuralı** şudur [PD3, s. 97]:

```
{P} S {Q},  P' ⇒ P,  Q ⇒ Q'
──────────────────────────────
        {P'} S {Q'}
```

Seçim (selection): `if B then S1 else S2` için kural [PD3, s. 98–99]:

```
{B and P} S1 {Q},  {(not B) and P} S2 {Q}
──────────────────────────────────────────
        {P} if B then S1 else S2 {Q}
```

Örnekte `if (x > 0) y = y - 1 else y = y + 1` ve sonşart `{y > 0}` alınır; then kısmı `{y > 1}`, else kısmı `{y > -1}` önşartını gerektirir; `{y > 1} ⇒ {y > -1}` olduğu için `{y > 1}` her iki dal için ortak önşarttır.
Mantıksal öntest döngüleri (while): Döngü sayısı belirli değilse önşartı hesaplamak kolay değildir. Döngü sayısı belirliyse, arka arkaya eklenmiş ifadeler dizisi gibi düşünülüp en son elemanın sonşartından geriye gidilir [PD3, s. 100–101]:

```
while y <> x do y = y + 1 end  {y = x}

0 iterasyon: {y = x}
1 iterasyon: {y = x - 1}
2 iterasyon: {y = x - 2}
3 iterasyon: {y = x - 3}
...
{y < x} ile 0 iterasyon birleşince önşart: {y <= x}
```

Sıra (sequence) kuralı [PD3, s. 103]:

```
{P1} S1 {P2},  {P2} S2 {P3}
──────────────────────────
      {P1} S1; S2 {P3}

y = 3*x + 1;  x = y + 3;  {x < 10}
2. ifadenin önşartı: y < 7
1. ifadenin önşartı: x < 2
```

Döngü kuralı ve değişmez (invariant) [PD3, s. 104–106]:

```
(I and B) S {I}
──────────────────────────────
{I} while B do S {I and (not B)}
```

`I` döngü sabiti (invariant) / tümevarımsal hipotezdir ve şu koşulları sağlamalıdır:
1. `P ⇒ I` — döngü başlangıcında `I` doğru olmalı.
2. `{I} B {I}` — Boolean hesabı `I`'nin doğruluğunu değiştirmemeli.
3. `{I and B} S {I}` — döngü gövdesi `I`'yı korumalı.
4. `(I and (not B)) ⇒ Q` — çıkışta `I` ve `not B` birlikte sonşartı gerektirmeli.
5. Döngü sonlanmalı (ispatı zor olabilir).

`I`, döngü sonşartının zayıflatılmış bir sürümü ve aynı zamanda bir önşarttır: döngü başlamadan önce yerine getirilebilecek kadar zayıf, çıkış şartıyla birleşince sonşartı zorlayacak kadar güçlü olmalıdır [PD3, s. 106].
Program ispatı: Bütün program için sonşart istenen sonuçtur; programda ilk ifadeye kadar geriye doğru çalışılır. Birinci ifadedeki önşart program şartnamesiyle (spec) aynıysa program doğrudur [PD3, s. 107]. Takas örneği [PD3, s. 108]:

```
{x = a and y = b}
t = x
x = y
y = t
{x = b and y = a}

y = t   {x = b and y = a} → {x = b and t = a}
x = y   {x = b and t = a} → {y = b and t = a}
t = x   {y = b and t = a} → {y = b and x = a}
```

Son adım, başlangıçtaki `{x = a and y = b}` önşartıyla aynıdır; program kanıtlanmış olur.
Aksiyomatik semantiğin değerlendirmesi [PD3, s. 109]: bir dildeki bütün ifadeler için aksiyom ve çıkarsama kuralları geliştirmek zordur; ispatların doğruluğu için iyi bir araç ve programlama mantığı için mükemmel bir çatıdır; ancak dil kullanıcıları ve derleyici yazanlar için kullanışlı değildir.

FONKSİYONEL (DENOTATİONAL) SEMANTİK

Denotasyonel semantik, **Scott ve Strachey (1970)** tarafından geliştirildi; **özyinelemeli fonksiyon teorisine** dayanır. Temel dildeki her varlık için hem bir **matematiksel nesne** hem de **fonksiyonel tanımlama** vardır. En soyut ve programların anlamını tanımlamada en yaygın kullanılan yöntemdir. Fonksiyon, bir varlığın örneklerini matematiksel modelin örnekleriyle eşleştirir; nesneler tanımlandığından, karşılık geldikleri varlıkların anlamlarını tanımlar [PD3, s. 110].
İkili sayı grameri [PD3, s. 110–111]:

```
<bin_num> → '0' | '1' | <bin_num> '0' | <bin_num> '1'
```

Anlam fonksiyonu `Mbin`, ikili sayıları negatif olmayan ondalık tamsayılarla eşler [PD3, s. 115]:

```
Mbin('0') = 0
Mbin('1') = 1
Mbin(<bin_num> '0') = 2 * Mbin(<bin_num>)
Mbin(<bin_num> '1') = 2 * Mbin(<bin_num>) + 1
```

Ondalık sayılar için `Mdec` benzerdir: `Mdec('0') = 0`, ..., `Mdec('9') = 9`; `Mdec(<dec_num> '0') = 10 * Mdec(<dec_num>)`, ..., `Mdec(<dec_num> '9') = 10 * Mdec(<dec_num>) + 9` [PD3, s. 117].
Deyimler `Z ∪ {error}` üzerine eşlenir; deyimler ondalık sayı, değişken veya iki operandlı ikili deyim olabilir [PD3, s. 118]:

```
Me(<expr>, s) =
  case <expr> of
    <dec_num>     => Mdec(<dec_num>, s)
    <var>         => if VARMAP(<var>, s) == undef then error
                     else VARMAP(<var>, s)
    <binary_expr> => if Me(left, s) == undef OR Me(right, s) == undef
                     then error
                     else if operator == '+' then Me(left, s) + Me(right, s)
                     else Me(left, s) * Me(right, s)
```

Programın durumu, bütün güncel değişkenlerin değerleridir: `s = {<i1, v1>, <i2, v2>, ..., <in, vn>}`. `VARMAP(ij, s) = vj`, bir değişkenin adı ve durumu verildiğinde güncel değerini döndürür [PD3, s. 114].
Atama durum kümelerini durum kümelerine eşler [PD3, s. 120]:

```
Ma(x := E, s) =
  if Me(E, s) == error then error
  else s' = {<i1', v1'>, ..., <in', vn'>}
       where for j = 1..n:
         vj' = VARMAP(ij, s)  if ij <> x
             = Me(E, s)       if ij == x
```

Mantıksal öntest döngüleri [PD3, s. 121–122]:

```
Ml(while B do L, s) =
  if Mb(B, s) == undef then error
  else if Mb(B, s) == false then s
  else if Msl(L, s) == error then error
  else Ml(while B do L, Msl(L, s))
```

Döngünün anlamı, program değişkenlerinin döngüdeki ifadeler belirtilen sayıda ve hatasız çalıştırıldıktan sonra aldığı değerlerdir. Esasında döngü, iterasyondan **özyinelemeye** dönüştürülmüştür; özyineleme, iterasyona göre matematiksel kesinlikle açıklaması daha kolaydır [PD3, s. 122].
Denotasyonel semantiğin değerlendirmesi [PD3, s. 123]: programların doğruluğunu ispatlamak için kullanılabilir; programlar hakkında düşünmek için sıkı (rigorous) bir yol sağlar; dil tasarımına yardımcı olabilir; derleyici üretme sistemlerinde kullanılmıştır; karmaşıklığı yüzünden dil kullanıcıları tarafından çok az kullanılmıştır.

> **KRITIK AYRIM**
>
> **İşlemsel / aksiyomatik / denotasyonel:** İşlemsel semantik anlamı **makine durumu ve geçişleriyle**, aksiyomatik semantik **mantıksal ön-son koşullarıyla**, denotasyonel semantik **matematiksel fonksiyonlarla** tanımlar. İlk ikisi daha uygulama yakın, üçüncüsü daha soyuttur; hiçbiri BNF'nin sözdizimindeki tekelini kuramamıştır.

YÖNTEMLERİN ÖZETİ

YÖNTEM	[[PROGRAM]]	GÜÇLÜ YANI	ZAYIF YANI
Operational	Soyut makine programı	Gerçekleştirmesi kolay	Muhakeme zor
Axiomatic	Özellikler kümesi	Teorem ispatı için iyi	Uygulamadan uzak
SOS	Geçiş sistemi	Eşzamanlılık ve non-determinizm için iyi	Eşitlik hakkında muhakeme zor
Denotational	Matematiksel ifade (tipik olarak fonksiyon)	Muhakemeyi kolaylaştırır	Uygun semantik alan bulmak zor

Kaynak: [PD3, s. 124].

SEMANTİK ÖZET ÖRNEK: FAKTÖRİYEL

Faktöriyel programı, üç yaklaşımın aynı programı nasıl farklı biçimlerde açıkladığını gösterir [PD3, s. 125–132]:

```
int fact(int x) {
  int z, y;
  z = 1;
  y = x;
  while (y > 0) {
    z = z * y;
    y = y - 1;
  }
  return z;
}
```

İşlemsel özet: `[x → 3]` başlangıç durumundan başlayarak her atama sonrası durum yazılır: `[x → 3, z → ⊥, y → ⊥]` → `[x → 3, z → 1, y → ⊥]` → `[x → 3, z → 1, y → 3]` → döngü adımlarıyla `z` ve `y` güncellenir → `[x → 3, z → 6, y → 0]` → dönüş değeri `6` [PD3, s. 125–129].
Denotasyonel özet: Faktöriyel fonksiyonu `f = λx. if x = 0 then 1 else x * f(x - 1)` olarak tanımlanır [PD3, s. 130].
Aksiyomatik özet: Döngü değişmezi `{x = n ∧ y > 0 ∧ z = n! / y!}` seçilir; her atama ve döngü adımı bu iddiayı korur; çıkışta `{x = n ∧ z = n!}` elde edilir [PD3, s. 131].
Operational özet: `for (e1; e2; e3)` döngüsü `e1; L1: if not(e2) goto L2; ...; e3; goto L1; L2: ...` biçimine çevrilir [PD3, s. 132].

STANDARTLAŞTIRMA

Bir dil yaygın olarak kullanılmaya başladıktan sonra, çeşitli donanım ortamlarına uyumlu gerçekleştirimlerin üretilebilmesi için dilin sözdizimi ve anlamı **tam ve açık** olarak tanımlanmalı ve bir **dil standardı** oluşturulmalıdır. Standart, dil tasarımcısı, destekleyen kuruluş veya ABD'de **ANSI**, uluslararası düzeyde **ISO** gibi bir kuruluş tarafından tanımlanabilir [PD3, s. 134].

BÖLÜM 3'ÜN SENTEZİ

PD3'ün ana çizgisi şudur [PD3, s. 133]:

- **BNF ve CFG** eşdeğer metadillerdir; programlama dillerinin sözdizimini tanımlamak için uygundurlar.
- **Özellik grameri**, bir dilin hem sözdizimini hem de statik anlamını tanımlayabilen biçimsel bir çerçevedir.
- **Anlam tanımlamanın birincil yöntemleri** işlemsel, aksiyomatik ve denotasyonel semantiktir; hiçbiri sözdizimindeki BNF kadar standart değildir.

C–G · ORTAK BÖLÜMLER
SÖZLÜK · PD3

C

Kavramlar ve tanımlar

PD3'ün sınav açısından kritik terimleri; sözdizimi, gramer, türetme ve anlam tanımlama yöntemleri.

SÖZDİZİMİ

Sözdizimi (syntax): Deyimlerin, ifadelerin ve program birimlerinin biçimsel yapısı. [PD3, s. 4]

Anlambilim (semantics): Aynı birimlerin anlamı. [PD3, s. 4]

Lexeme: Dilin en alt düzey sözdizimsel birimi. [PD3, s. 9, 11]

Token: Lexeme'lerin kategorisi (identifier, keyword, symbol). [PD3, s. 9, 11]

Soyut sözdizimi: Anlamlı bileşenleri gösteren, gösterimden bağımsız yapı. [PD3, s. 7–8]

Ayrıştırma ağacı (parse tree): Türetmenin hiyerarşik gösterimi; yapraklar terminal. [PD3, s. 48]

GRAMER

Metadil: Başka bir dilin sözdizimini anlatmak için kullanılan dil. [PD3, s. 22]

BNF: Backus–Naur Form; CFG'ye eşdeğer metadil. [PD3, s. 31]

CFG: `A → γ` kurallarıyla tanımlanan, PDA ile kabul edilen dil sınıfı. [PD3, s. 30]

Terminal / terminal olmayan: Atomik sembol / kurallarla tanımlanan ara sembol. [PD3, s. 33]

Türetme (derivation): Başlangıç sembolünden cümleye kuralların uygulanması. [PD3, s. 38]

Cümlesel biçim / cümle: Türetmedeki her sembol dizisi / yalnızca terminallerden oluşan biçim. [PD3, s. 45]

Belirsizlik (ambiguity): Aynı cümlesel biçim için iki veya daha fazla ayrıştırma ağacı. [PD3, s. 51]

EBNF: `[ ]`, `{ }`, `|` ile BNF'i kısaltan genişletilmiş form. [PD3, s. 55–59]

ANLAM

Özellik grameri (AG): CFG + özellik kümeleri + semantik fonksiyonlar + predicate'ler. [PD3, s. 64]

Sentezlenmiş özellik: Çocuklardan ebeveyne akan, `S(X0)=f(...)` ile tanımlanan özellik. [PD3, s. 65]

Miras alınmış özellik: Ebeveynden çocuklara akan, `I(Xj)=f(...)` ile tanımlanan özellik. [PD3, s. 65]

İşlemsel semantik: Anlamı soyut makine durumu ve geçişleriyle açıklama. [PD3, s. 82]

SOS: `(C, ⇒, I, F, O)` beş-tuple'ı ile yapısal işlemsel semantik. [PD3, s. 85]

Aksiyomatik semantik: Önşart/sonşart ve çıkarsama kurallarıyla anlam tanımlama. [PD3, s. 92]

En zayıf önşart: Sonşartı garanti eden asgari önşart. [PD3, s. 93]

Döngü değişmezi (invariant): Döngü boyunca doğru kalan iddia. [PD3, s. 104]

Denotasyonel semantik: Anlamı matematiksel nesne/fonksiyonlarla tanımlama. [PD3, s. 110]

VARMAP: Değişken adı ve durumdan güncel değeri döndüren fonksiyon. [PD3, s. 114]

SIK KARIŞTIRILAN AYRIMLAR

Sözdizimi / anlambilim: Biçim / anlam; ikisi birlikte dil tanımını verir.

Tanıyıcı / üreteç: String'in dilde olup olmadığına karar verir / dilin cümlelerini üretir. [PD3, s. 20]

Soyut / metinsel sözdizimi: Anlamlı bileşenlerin ağacı / karakter dizisi gösterimi. [PD3, s. 7–10]

Belirsiz gramer / belirsiz dil: Gramerin özelliği / dilin değil, gramerin belirsizliği. [PD3, s. 51]

D

Yöntemler ve süreçler

Sözdizimi tanımlama, türetme, özellik değerlendirme ve anlam tanımlama adımlarının sıralı özeti.

D.1 SÖZDİZİMİ TANIMLAMA AKIŞI

1. **Alfabe ve lexeme'leri belirle:** Hangi karakterler, hangi token kategorileri? [PD3, s. 9–11]
2. **Terminal sembolleri listele:** `+`, `-`, `if`, `>=` gibi atomik birimler. [PD3, s. 33]
3. **Terminal olmayanları tanımla:** `<program>`, `<stmt>`, `<expr>` gibi ara semboller. [PD3, s. 33]
4. **Kuralları yaz:** Sol taraf terminal olmayan, `→` / `:=`, sağ taraf sembol dizisi. [PD3, s. 34]
5. **Başlangıç sembolünü seç:** Dilin ana elemanı. [PD3, s. 37]
6. **Türetme ile doğrula:** Başlangıçtan cümleye adım adım in. [PD3, s. 38–44]
7. **Belirsizliği denetle:** Aynı cümle için iki ağaç var mı? [PD3, s. 51–54]
8. **Gerekirse EBNF'ye çevir:** `[ ]`, `{ }`, `|` ile kısalt. [PD3, s. 55–61]

D.2 AYRIŞTIRMA AĞACI OKUMA

1. Kök, başlangıç sembolüdür.
2. Her iç düğüm bir terminal olmayan; çocukları o kuralın sağ tarafıdır.
3. Yapraklar terminal sembollerdir.
4. Ağacın derinliği, operatör önceliğini ve birleşirliği gösterir.
5. Aynı cümle için iki farklı ağaç, gramerin belirsiz olduğunu gösterir. [PD3, s. 48–54]

D.3 ÖZELLİK GRAMERİ DEĞERLENDİRME

1. Her sembol için özellik kümesini tanımla (`actual_type`, `expected_type`). [PD3, s. 64]
2. Her kural için semantik fonksiyonları yaz. [PD3, s. 65]
3. Predicate'leri ekle (tip uyumu, ad eşleşmesi). [PD3, s. 64, 67]
4. Sentezlenmiş özellikler için aşağıdan yukarıya; miras alınanlar için yukarıdan aşağıya ilerle. [PD3, s. 69]
5. İki yönlü akış gerekiyorsa düzenleri birleştir. [PD3, s. 69]

D.4 ANLAM TANIMLAMA YÖNTEMLERİ

YÖNTEM	GİRDİ	ÇIKTI	TİPİK KULLANIM
Operational	Program + soyut makine	Konfigürasyon geçişleri	Dil el kitapları, yorumlayıcı
SOS	Program + geçiş sistemi	`(C, ⇒, I, F, O)`	Eşzamanlılık, non-determinizm
Axiomatic	Program + ön/son şart	İspat yükümlülükleri	Program doğrulama
Denotational	Program + matematiksel alan	Fonksiyon	Derleyici üretimi, dil tasarımı

E

Formüller, kurallar ve olgular

PD3'te geçen gramer gösterimleri, çıkarsama kuralları ve sayısal örnekler. Yalnızca kaynakta verilen bilgiler; ek açıklamalar etiketlidir.

E.1 BNF / EBNF GÖSTERİMLERİ

```
# BNF temel biçimi
<nonterminal> → terminal | <nonterminal> terminal

# Özyinelemeli liste
<tanımlayıcı_listesi> := tanımlayıcı | tanımlayıcı, <tanımlayıcı_listesi>

# EBNF
<proc_call> → ident [ "(" <expr_list> ")" ]
<term>      → <term> ( "+" | "-" ) const
<ident>     → letter { letter | digit }
```

E.2 AYRIŞTIRMA VE BELİRSİZLİK

```
Belirsiz:
<expr> → <expr> <op> <expr> | const
<op>   → / | -

Belirli (öncelikli):
<expr> → <expr> - <term> | <term>
<term> → <term> / const | const

Sol birleşirli:
<expr> → <expr> + const | const
```

E.3 AKSİYOMATİK SEMANTİK KURALLARI

```
Atama:      {Q_{x→E}}  x = E  {Q}
Sonuç:      {P} S {Q}, P' ⇒ P, Q ⇒ Q'  /  {P'} S {Q'}
Seçim:      {B and P} S1 {Q}, {(not B) and P} S2 {Q}
            ──────────────────────────────────────
            {P} if B then S1 else S2 {Q}
Sıra:       {P1} S1 {P2}, {P2} S2 {P3}  /  {P1} S1; S2 {P3}
Döngü:      (I and B) S {I}  /  {I} while B do S {I and (not B)}
```

E.4 SAYISAL ÖRNEKLER VE OLGULAR

OLGU	DEĞER / İÇERİK	KAYNAK
BNF'nin doğuşu	1959, Backus, ALGOL 58	PD3, s. 31
CFG'nin doğuşu	1950'lerin ortası, Chomsky	PD3, s. 30
Denotasyonel semantik	Scott ve Strachey, 1970	PD3, s. 110
`a = b + 1 {a > 1}` en zayıf önşart	`{b > 0}`	PD3, s. 94
`x = 2*y - 3 {x > 25}` en zayıf önşart	`{y > 14}`	PD3, s. 95
`while y <> x do y = y + 1 end {y = x}` önşart	`{y <= x}`	PD3, s. 101
SOS beş-tuple	`C, ⇒, I, F, O`	PD3, s. 85
RVM konfigürasyonu	`Komutlar × PC × RegisterFile`	PD3, s. 86

E.5 KAYNAK İÇİ TUTARSIZLIKLAR VE YAZIM NOTLARI

— **"İçerik bağımsız" / "bağlam-bağımsız":** Kaynakta `context-free` için "içerik bağımsız" kullanılır; yaygın Türkçe terim **bağlam-bağımsız**'dır. Kılavuzda bağlam-bağımsız yeğlenmiş, kaynak terimi parantezde korunmuştur [PD3, s. 22–23].

— `<proc_def>` **örneği:** Kaynakta `procedure <proc_name>[1]` biçimindeki köşeli parantezler özellik oluşumlarını gösterir; bunlar BNF'nin terminal olmayan parantezleriyle karıştırılmamalıdır [PD3, s. 67].

— **Faktöriyel özeti:** Kaynak slaytlarda durum gösterimi birkaç sayfaya yayılmıştır; kılavuzda tek akışta birleştirilmiştir [PD3, s. 125–132].

— **Ödev slaytı (s. 135):** Dört araştırma ödevi vardır; bunlar sınav kapsamı değildir ve kılavuza alınmamıştır.

C–G · ORTAK BÖLÜMLER
BÖLÜMLER ARASI AĞ

F

Bağlantılar ve karşılaştırmalar

PD3'ün PD2 ve sonraki bölümlerle bağlantısı; sözdizimi ve anlambilim konularının dersin bütününe nasıl oturduğu.

F.1 PD2 → PD3: TARİHTEN BİÇİMSEL TANIMA

PD2'DEKİ ÇIKIŞ NOKTASI	PD3'TEKİ KARŞILIĞI	İLİŞKİ
ALGOL 60'ın BNF ile tanımlanması (PD2 2.5)	BNF'nin dört bileşeni ve türetme (PD3 3.3)	Tarihsel olay, biçimsel yöntemin doğuşudur.
Derleyici ön ucu (PD2 2.3, PD0 0.4)	Sözcüksel analiz, token, sembol tablosu (PD3 3.2)	Ön ucun ilk iki aşaması burada ayrıntılanır.
Tip kavramı (PD2 2.5, ALGOL 58)	Özellik gramerleri ve tip denetimi (PD3 3.4)	Tip, gramere özellik olarak eklenir.
LISP'in kod–veri aynılığı (PD2 2.4)	Soyut sözdizimi ve ağaç (PD3 3.1)	Kodun yapısal gösterimi, anlam tanımlamanın önkoşuludur.

F.2 SONRAKİ BÖLÜMLERLE BAĞLANTI

- **PD4 (sözcüksel/sözdizimsel analiz):** PD3'ün tanıyıcı/üreteç ayrımı ve CFG tanımı, PD4'ün ayrıştırma algoritmalarına (yukarıdan aşağıya, aşağıdan yukarıya) zemin hazırlar.
- **PD5 (adlar, bağlama, kapsam):** Özellik gramerlerindeki `look-up` ve `expected_type`, isim çözümleme ve kapsam kurallarının başlangıcıdır.
- **PD6 (veri tipleri):** Tip uyumluluğu ve statik semantik, veri tipi sistemlerinin biçimsel temelidir.
- **PD7 (kontrol yapıları):** `if`, `while` ve sıra kuralları, aksiyomatik semantikte doğrudan karşılık bulur.
- **PD8 (altprogramlar):** Ada prosedür adı örneği ve parametre geçişi, altprogram semantiğine bağlanır.

> **BU KILAVUZUN YORUMU**
>
> PD3'ün ortak ekseni **tanımlama düzeyidir**: sözdizimi "hangi biçim", statik semantik "hangi tip/tanım", dinamik semantik "hangi durum geçişi" sorusunu yanıtlar. BNF'nin başarısı, sözdizimini **sonlu kurallarla** tanımlamasıdır; anlambilimin tek bir formalizme ulaşamaması ise anlamın **bağlama ve duruma** bağlı olmasındandır. Bu yüzden üç semantik yöntem birbirini dışlamaz; farklı sorulara farklı soyutlama düzeyinde yanıt verir.

G

Kapsam denetimi

Bu bölüm, PD3'ün hangi sayfalarının kapsandığını, hangi düşük metinli slaytların görsel olarak incelendiğini ve neyin bilinçli olarak dışarıda bırakıldığını raporlar.

G.1 Kaynak bazında kapsam

KAYNAK	SLAYT	KAPSANAN BÖLÜMLER VE SAYFA ARALIKLARI	DURUM
PD3	136	3.1 (s. 3–10) · 3.2 (s. 11–29) · 3.3 (s. 30–62) · 3.4 (s. 63–77) · 3.5 (s. 78–134) · özet (s. 133) · ödev (s. 135) · kaynaklar (s. 136)	Tam kapsam

G.2 Metin katmanı olmayan / düşük metinli slaytlar

PD3'te **18** düşük metinli slayt vardır: **1, 5, 19, 26, 39, 42, 46, 60, 73, 74, 76, 77, 80, 89, 111, 128, 129, 130**. Bunlar şema, ağaç, tablo veya boş slayt türündedir. Aşağıdaki tablo, bu slaytların nasıl ele alındığını özetler.

SAYFA	İÇERİK / GÖRSEL	KULLANIM
1	Bölüm kapağı	İçindekilere ve bölüm açılışına yansıtıldı
5	Sözdizimi–anlam ilişkisi şeması	3.1 metniyle açıklandı
19	`Array<Array<int>>` örneği	`>>` belirsizliği olarak metne alındı
26	Hata türleri tablosu	Hata türleri tablosuyla aktarıldı
39, 42, 46	Gramer/türetme şemaları	Türetme örnekleri metin ve kod bloklarıyla verildi
60	EBNF özeti	EBNF tablosu ve metasembol açıklaması
73, 74	Nitelik akışı / tam bağlanmış ağaç	3.4 değerlendirme sırası metniyle
76, 77	Sözdizim grafikleri	3.3 sonunda kavramsal açıklama
80	Anlam tanımlama şeması	3.5 girişinde üç yaklaşım
89	Geçiş kuralı formu	SOS beş-tuple ve RVM ile
111	`110` için ayrıştırma ağacı	İkili sayı örneğiyle
128, 129, 130	Faktöriyel durum özetleri	Faktöriyel örneğinde birleştirildi

G.3 Bilinçli dışarıda bırakılanlar ve sonuç

— **Bölüm kapağı ve konu listesi** (s. 1–2) içindekilere yansıtıldı.

— **Ödev slaytı** (s. 135): BNF/EBNF araştırması, Java BNF'i, `a=a*(b+(c*a))` türetmesi ve semantik yöntemler ödevi. Bunlar sınav sorusu değil, kaynakta verilen araştırma görevleridir; kılavuzda yalnızca kapsam notu olarak anılmıştır.

— **Tekrarlanan "Lexical / Syntax / Semantic" hattı** (s. 27–29) bir kez, 3.2 sonunda özetlendi.

— **Uzun kod parçaları ve tam ayrıştırma ağacı çizimleri** (s. 111, 128–130) birebir çoğaltılmadı; ayırt edici yapı ve sonuçlar verildi.

— **Dekoratif görseller** yeniden üretilmedi; tekrar eden kaynakça listesi (s. 136) kapsam dışı bırakıldı.

Sonuç: PD3'ün **5/5 alt bölümü** ve **136/136 slaytı** kapsandı. Düşük metinli 18 slaytın tamamı incelendi; erişilemeyen sayfa tespit edilmedi. Otomatik başlık denetimi `build --check` ile, tablo/şekil/örnek denetimi bu bölümde elle yapılmıştır.